

## 퀄리저널 100% 성공 배포 마스터 워크플로우 가이드

QualiJournal 운영자와 개발자를 위한 커밋 → 푸쉬 → 배포 마스터 워크플로우 가이드입니다. 각 단계를 따라하면 CI 파이프라인부터 Cloud Run 서비스 배포까지 **한 번에 성공**할 수 있습니다. 섹션별로 핵심 내용을 정리하고, PowerShell용 코드 블록을 제공하여 초보자도 쉽게 따라할 수 있도록 구성했습니다.

### 1. Git 서명 커밋

모든 Git 커밋에는 **서명(Sign)**을 포함하여 GitHub에서 “Verified” 배지를 받도록 합니다. 이는 배포 파이프라인의 신뢰성과 보안을 높이고, 브랜치 보호 규칙에 따라 **서명되지 않은 커밋이 main에 머지되는 것을 차단**할 수 있습니다. SSH 기반 서명키를 생성하고 설정하는 방법은 다음과 같습니다:

- **SSH 서명키 생성 (PowerShell):** 최신 Git은 GPG 외에 **SSH 키로 커밋 서명**을 지원합니다. 관리자 권한 PowerShell을 열고 OpenSSH 클라이언트가 설치되어 있는지 확인합니다 (Windows 10+에서는 기본 제공). 그런 다음 ED25519 알고리즘으로 서명용 SSH 키를 생성합니다 (`-C` 코멘트는 키 식별용):

```
# PowerShell: SSH 서명키 생성 (ed25519 알고리즘)
ssh-keygen -t ed25519 -N "" -C "QualiJournal Git Signing Key" -f
"$env:USERPROFILE\.ssh\id_ed25519_signing"
```

위 명령을 실행하면 `id_ed25519_signing` (개인키)과 `id_ed25519_signing.pub` (공개키) 파일이 생성됩니다. `-N ""` 옵션은 패스프레이즈 없이 키를 생성하는 것으로, 편의성을 위해 사용했습니다 (패스프레이즈를 설정하는 경우 이후 커밋시 에이전트(ssh-agent)에 추가해야 함).

- **Git 전역 설정:** 생성한 SSH 키를 커밋 서명에 사용하도록 Git을 설정합니다. Git 설정에서는 GPG 대신 SSH를 서명 도구로 사용하도록 하고, 방금 만든 **공개키**를 서명키로 등록합니다. 또한 모든 커밋에 서명을 포함하도록 기본 설정합니다:

```
git config --global gpg.format ssh
git config --global user.signingkey "$env:USERPROFILE\.ssh\id_ed25519_signing.pub"
git config --global commit.gpgsign true
```

위 설정으로 Git은 커밋 서명에 SSH 키를 사용하게 됩니다 <sup>1</sup>. `user.signingkey`에는 공개키 파일 경로나 키 이름을 지정할 수 있습니다. (`commit.gpgsign true`로 기본적으로 모든 커밋에 서명이 붙습니다.)

- **GitHub에 서명키 등록:** GitHub에 방금 만든 공개키를 등록해야 **서명된 커밋이 "Verified"**로 확인됩니다. GitHub 웹의 **Settings > SSH and GPG keys**에서 “**New SSH key**”를 클릭하고, 키 타입을 Signing으로 선택한 뒤 공개키(`id_ed25519_signing.pub`) 내용을 붙여넣습니다 <sup>2</sup> <sup>3</sup>. **GitHub CLI**를 사용한다면 다음 명령으로 동일하게 등록 가능합니다 <sup>4</sup>:

```
gh ssh-key add $env:USERPROFILE\.ssh\id_ed25519_signing.pub --title "QualiJournal Signing Key" --type signing
```

위 명령으로 GitHub 계정에 서명용 SSH 공개키가 추가됩니다. (동일한 SSH 키를 Git 접속용과 서명용으로 모두 쓰려면 각각 한 번씩 업로드해야 합니다 <sup>5</sup>.)

- **서명 커밋 작성:** 이제 커밋할 때 `-S` 옵션을 주면 해당 SSH 키로 서명됩니다. 예를 들어 새로운 기능을 추가했다면 다음과 같이 커밋합니다:

```
git add .
git commit -S -m "feat: 새로운 기능 추가"
```

커밋 후 `git log --show-signature -1`로 서명 정보를 확인하거나, GitHub 리포지토리에서 해당 커밋에 “Verified” 배지가 표시되는지 확인하세요. **브랜치 보호**에서 “Require signed commits”를 활성화하면, 서명 검증된 커밋만 main 브랜치에 머지되도록 강제할 수 있습니다.

## 2. 브랜치 전략 및 보호 규칙

모든 작업은 main 브랜치를 보호하고, 기능 개발과 버그 수정을 위해 **기능 브랜치(feature branch)**를 사용하는 전략을 따릅니다:

- **브랜치 네이밍 규칙:** 새로운 기능 작업 시 브랜치를 `feat-` 접두사로, 버그 수정 작업 시 `fix-` 접두사로 시작해 만듭니다. 예를 들어 UI 개선 기능 작업은 `feat-ui-improve`, API 오류 수정 작업은 `fix-api-error`와 같이 짓습니다. 브랜치 이름으로 작업 내용을 쉽게 유추할 수 있도록 **짧고 명확하게** 작성하세요.
- **Pull Request 통한 머지:** 메인 저장소에 직접 푸쉬하지 말고, 각 작업은 기능 브랜치에서 커밋한 후 **Pull Request(PR)**를 생성해 merge합니다. PR을 만들 때 적절한 설명을 달고, 리뷰어를 지정합니다. 최소 **1인 이상의 코드 리뷰 승인**과 **CI 체크 통과** 후에만 main에 병합합니다. 이 과정을 통해 코드 품질과 빌드 안정성을 확보합니다.
- **브랜치 보호 설정:** GitHub **Settings > Branches**에서 main 브랜치에 대한 보호 규칙을 설정합니다. “Require pull request reviews” 옵션을 켜고, “Require status checks to pass before merging” 옵션을 활성화합니다. 필요한 Status Check으로 CI 파이프라인의 배포 워크플로우 (예: “Deploy to Cloud Run”)를 지정합니다. 이렇게 하면 GitHub Actions의 **배포 확인 체크**가 통과되지 않으면 PR을 머지할 수 없습니다. 또한 “Include administrators” 옵션을 켜서 관리자의 실수로도 보호 규칙을 어기지 않도록 하고, **강제 푸쉬 금지** 등도 활성화합니다.  
(Tips: main 브랜치 보호 규칙에 “Require signed commits”를 추가하면, 서명되지 않은 커밋이 포함된 PR도 머지가 차단되어 보안을 한층 강화할 수 있습니다.)

## 3. GitHub Actions를 이용한 자동 배포

메인 브랜치에 푸쉬되면 **GitHub Actions CI**가 자동으로 실행되어 Cloud Run에 새로운 리버전을 배포합니다. QualiJournal 리포지토리에는 이미 설정된 워크플로우가 있으며, 주요 동작은 다음과 같습니다:

- **CI/CD 워크플로우 트리거:** `main` 브랜치로 푸쉬될 때 **CI-CD Cloud Run** 액션이 시작됩니다 <sup>6</sup>. 이 워크플로우는 애플리케이션을 빌드하고 새 이미지를 Artifact Registry에 푸쉬한 뒤, Cloud Run에 배포합니다 <sup>7</sup>.
- **자동 Cloud Run 배포:** GitHub Actions에서 Google Cloud에 인증한 후, Docker 이미지를 빌드/푸쉬하고 `gcloud run deploy` 명령어로 Cloud Run 서비스를 업데이트합니다. QualiJournal Admin 서비스의 **Cloud Run 서비스 이름**은 `quali-admin-domap`으로 고정되어 있습니다 (도메인 매핑이 이 서비스에 연결되어 있기 때문입니다). 최신 워크플로우에서는 소스 코드로 빌드 배포를 수행하기도 합니다 <sup>8</sup>. 배포 완료

후 곧바로 `gcloud run services update-traffic ... --to-latest`를 호출하여 **새 리비전으로 트래픽을 100% 이동**시킵니다 <sup>8</sup>. 이 단계 덕분에 배포 직후 곧바로 신규 리비전이 서비스에 반영됩니다 (이전 리비전에 트래픽이 남아있지 않아야 함).

- **필수 환경변수 설정:** 배포 시 애플리케이션이 필요로 하는 환경변수를 설정해야 합니다. 주요 환경변수로 `ADMIN_TOKEN`, `API_KEY`, `DB_URL` 등이 있습니다. CI 워크플로우에서 `ADMIN_TOKEN`은 **시크릿 매니저(Secret Manager)**와 연계되어 주입됩니다 (GitHub Actions에서 Cloud Run 배포 시 `--set-secrets "ADMIN_TOKEN=ADMIN_TOKEN:latest"` 형태로 지정) <sup>9</sup>. 이를 위해 GCP의 Secret Manager에 `ADMIN_TOKEN` 값을 미리 저장해두고, GitHub Actions에 해당 권한 및 시크릿 참조를 설정해 둡니다. 반면 `API_KEY`나 `DB_URL` 등은 민감도에 따라 GitHub 리포지토리의 **Secrets**에 저장하고, 배포 명령에 `--set-env-vars` 또는 `env_vars`로 전달합니다 <sup>10</sup>. 예를 들어 워크플로우에서 다음과 같이 설정합니다:

```
flags: >-
--allow-unauthenticated
--remove-env-vars ADMIN_TOKEN
--set-secrets ADMIN_TOKEN=ADMIN_TOKEN:latest
env_vars: >-
API_KEY=${{ secrets.API_KEY }},
DB_URL=${{ secrets.DB_URL }}
```

위 설정으로 Cloud Run 서비스 배포 시 `ADMIN_TOKEN`은 시크릿에서 가져오고, `API_KEY`와 `DB_URL`은 GitHub Secrets 값을 일반 환경변수로 주입하게 됩니다 <sup>9</sup>. **ADMIN\_TOKEN**은 어드민 API의 인증에 쓰이는 토큰이고, **API\_KEY**는 외부 API 연동 키 등이 해당됩니다. **DB\_URL**은 데이터베이스(예: Cloud SQL)의 연결 문자열로, 애플리케이션이 필요로 하면 설정합니다. (만약 Cloud SQL 등의 구성을 사용한다면, 별도의 Cloud Run 시크릿 또는 VPC 연결 설정도 필요할 수 있습니다.)

- **배포 완료 및 스모크 테스트:** Actions 파이프라인은 배포 후 Cloud Run의 URL을 확인하고 간단한 **스모크 테스트**를 수행합니다. 예를 들어 `/health` 엔드포인트에 접속해 200 OK 응답을 확인하고, `/api/status`에 **토큰 없이** 호출해 401(Unauthorized)이 나오는지, 토큰을 포함해 호출해 200이 나오는지, `/api/report`가 200을 응답하는지를 자동으로 확인합니다 <sup>11</sup>. 이러한 1차 **건강 체크**를 통과하면 배포가 완료되며, PR의 Required Check도 통과하게 됩니다.

## 4. 도메인 매핑 & 캐시 무효화

QualiJournal Admin 서비스는 **커스텀 도메인인** `admin.standardai.co.kr`로 접근합니다. 배포 후 새 리비전이 제대로 서비스되려면 도메인 매핑과 브라우저 캐시를 고려해야 합니다:

- **도메인 매핑 확인:** `admin.standardai.co.kr` 도메인이 최신 Cloud Run 서비스 리비전에 연결되어 있어야 합니다. Cloud Run에서는 도메인 매핑이 특정 서비스의 라우트에 연결되는데, 일반적으로 도메인 매핑은 한 번 설정하면 변경하지 않으므로 배포마다 손댈 필요는 없습니다. 다만 **예상 서비스에 바인딩되어 있는지** 확인이 필요할 때가 있습니다. GCP 콘솔의 Cloud Run > Domain mappings 메뉴에서 해당 도메인이 `quali-admin-domap` 서비스로 연결되어 있는지 확인하세요. CLI로 확인하려면 아래 명령을 사용할 수 있습니다:

```
gcloud beta run domain-mappings describe --domain "admin.standardai.co.kr" --region "asia-northeast1" --format="value(spec.routeName)"
```

위 명령 결과가 `quali-admin-domap` 을 가리키면 올바르게 매핑되어 있는 것입니다 <sup>12</sup> . 만약 다른 서비스나 빈 값이 나온다면, 도메인 매핑 설정을 수정해야 합니다.

- **브라우저 캐시 무효화(Cache Invalidation)**: 배포 후 프론트엔드 정적 파일이나 서비스 워커(Service Worker)가 이전 버전을 캐싱하고 있어 **화면에 변화가 안 나타나는** 경우가 있을 수 있습니다. 이를 해결하려면 사용 중인 브라우저 캐시를 무효화해야 합니다. 방법: **강력 새로고침** (크롬의 경우 `Ctrl + F5` 또는 `Shift + Reload`)을 하거나, URL에 임시 쿼리 파라미터를 추가하여 요청합니다. 예를 들어 `https://admin.standardai.co.kr/?v=12345` 같이 `?v=` 뒤에 랜덤 숫자를 붙여 접속하면, 새로운 주소로 인식되어 캐시를 우회할 수 있습니다. 정적 자원(js/css)에 캐시 해시가 적용되어 있다면 기본 새로고침으로도 최신 파일이 로드되지만, 수동으로 캐시를 무효화하면 확실합니다.

Tech Tips: QualiJournal 배포 과정에서는 **서비스 워커**에 커밋 해시를 삽입하고 정적 파일 이름에 해시를 붙여 빌드함으로써 캐시 이슈를 줄이고 있습니다 <sup>13</sup> <sup>14</sup> . 그래도 브라우저가 오래된 내용물을 보여줄 때는 위와 같이 쿼리스트링을 추가하거나, 개발자 도구에서 캐시를 비우고 새로고침하세요.

## 5. 배포 실패 원인 & 해결

간혹 배포가 완료되었어도 변경 사항이 반영되지 않거나, 서비스 응답이 예상과 다를 때가 있습니다. 아래는 **주요 실패 원인**과 **해결 방법**입니다:

- **문제: 패치 적용이 안 된 것처럼 보임** - 새 기능을 배포했는데도 웹 화면이나 API 결과에 변화가 없는 경우입니다.  
**원인**: Cloud Run 서비스가 여전히 이전 리비전을 제공하거나, 브라우저 캐시로 인해 구버전 자원이 로드되고 있을 수 있습니다. 간혹 GitHub Actions 워크플로우 실패로 배포가 스킵되었을 가능성도 있습니다.  
**해결**: GCP 콘솔의 Cloud Run 페이지에서 해당 서비스의 **Revisions** 탭을 확인합니다. 최신 커밋 해시를 포함한 리비전이 생성되었는지, 그리고 **트래픽(traffic)**이 100% 할당되었는지 봅니다. 만약 새 리비전이 보이는데 **ROUTING** 컬럼에 `(0%)` 또는 일부만 할당된 경우, 수동으로 최신 리비전에 트래픽을 모두 보내야 합니다. 이를 위해 다음 명령을 실행합니다:

```
gcloud run services update-traffic quali-admin-domap --region asia-northeast1 --to-latest
```

이 명령은 해당 서비스의 최신 리비전에 트래픽을 100% 몰아줍니다 <sup>8</sup> . 그런 다음 **섹션 4**의 방법으로 브라우저 캐시를 무효화하고 새로고침하여 변경사항이 나타나는지 확인합니다. 여전히 반영되지 않는다면, GitHub Actions 로그를 확인해 배포 실패 여부를 확인하고, 필요 시 워크플로우를 재실행하거나 수동 배포를 검토합니다.

- **문제: API 응답 이상 (401 또는 오류)** - 배포 후 API 요청이 예상과 다른 HTTP 상태 코드를 반환하는 경우입니다. 예를 들어 `/api/status` 나 `/api/report` 를 호출했을 때 **401 Unauthorized**가 발생한다면, 인증 토큰 이슈일 수 있습니다.  
**원인**: ADMIN\_TOKEN 또는 API\_TOKEN 환경변수가 설정되면서 해당 API가 **보호 모드**로 전환된 상황입니다. QualiJournal의 규칙에 따르면 ADMIN\_TOKEN(API 관리용 토큰)이 설정되면 모든 보호된 API는 올바른 토큰이 있을 때만 200 OK를返します <sup>15</sup> <sup>16</sup> . 401이 나오는 것은 **토큰을 포함하지 않았거나 잘못된 토큰**을 보냈다는 뜻입니다 (배포 시 사용된 ADMIN\_TOKEN과 테스트에 사용한 토큰 값 불일치 가능성).  
**해결**: 우선 **API 호출에 토큰을 포함**해보세요. `/api/status` 의 경우 Authorization 헤더에 `Bearer <ADMIN_TOKEN>` 을 넣어 200 응답을 얻을 수 있는지 확인합니다 <sup>17</sup> . 만약 토큰을 넣었는데도 401이 지속 된다면, 사용 중인 토큰 값이 GitHub 시크릿 및 GCP Secret Manager에 저장된 값과 같은지 확인해야 합니다. 만약 ADMIN\_TOKEN을 변경했다면, Secret Manager와 GitHub Secrets 양쪽에 모두 최신 값으로 업데이트 하고 다시 배포해야 합니다.

다른 API에서 5xx 에러가 발생한다면 애플리케이션 내부 오류일 수 있습니다. 이때는 Cloud Run 로그를 확인합니다.  
**Cloud Run 로그 확인**: GCP 콘솔의 로그 뷰어에서 `quali-admin-domap` 서비스의 로그를 확인하거나, gcloud

CLI로 `gcloud logs read --project=<PROJECT_ID> --service=quali-admin-domap` 명령을 사용합니다. `ERROR` 레벨의 로그나 예외 스택트레이스를 찾아 원인을 파악하세요. 예를 들어 `/api/report` 호출 시 500 오류가 난다면, 관련 로직에서 exception이 발생했을 수 있으므로 로그에 남은 에러 메시지를 확인해야 합니다. 데이터베이스 연결 오류라면 DB\_URL 설정 문제일 수 있고, 외부 API 오류라면 API\_KEY 문제일 수 있습니다.

- **문제: 배포 스텝 자체의 실패** – GitHub Actions에서 배포 작업(Job)이 빨간불로 실패하는 경우입니다.  
**원인:** Docker 빌드 에러, GCP 인증 오류, Cloud Run 배포 커맨드 오류 등이 있을 수 있습니다. 예를 들어 Secret Manager 권한 문제로 `--set-secrets` 단계가 실패하거나, Cloud Run 서비스 업데이트 시 리소스 한도 초과 등이 발생할 수 있습니다.  
**해결:** GitHub Actions의 **Logs**를 열어 어떤 스텝에서 에러가 났는지 확인합니다. 권한/인증 이슈일 경우 GCP 서비스 계정 및 WIF 설정을 재점검하고, 빌드 실패라면 Dockerfile이나 코드 오류를 수정해야 합니다. CI 단계에서 문제를 해결한 후 커밋을 다시 푸쉬하면 워크플로우가 재실행되어 배포를 시도합니다. 임시로 배포해야 한다면 로컬에서 `gcloud run deploy` 명령으로 수동 배포하고, 이후 CI 설정을 바로잡습니다.

## 6. 초심자를 위한 복붙용 스크립트 모음

아래는 각 단계를 **복사-붙여넣기**로 실행할 수 있는 예시 스크립트입니다. 숙련자뿐 아니라 Git/GCP에 익숙하지 않은 초심자도 따라 할 수 있도록 간단한 명령으로 구성했습니다.

- **Git 커밋 & 푸쉬 (서명 포함)** – 수정 사항을 커밋하고 원격 저장소에 올리는 과정:

```
git add -A # 변경된 모든 파일 스테이징
git commit -S -m "fix: 문제된 API 응답 수정" # 서명하여 커밋(-S)
git push origin fix-api-response-bug # 현재 브랜치를 원격에 푸쉬
```

위 예에서는 `fix-api-response-bug` 이름의 브랜치를 사용했습니다. 원격에 푸쉬한 후 GitHub에서 PR을 생성해 코드 리뷰 및 머지를 진행하세요.

- **Cloud Run 최신 리비전 배포 및 스모크 테스트 (PowerShell)** – 배포 직후 혹은 수동 점검 시 활용할 수 있는 PowerShell 스크립트입니다:

```
# 1) 최신 리비전으로 트래픽 100% 이동
gcloud run services update-traffic quali-admin-domap --region asia-northeast1 --to-latest

# 2) 캐시 무효화를 위해 기본 페이지에 랜덤 쿼리 파라미터를 붙여 브라우저 열기
Start-Process "https://admin.standardai.co.kr/?cache=$(Get-Random)"

# 3) 주요 엔드포인트 스모크 테스트 (HTTP 상태 코드 확인)
(Invoke-WebRequest "https://admin.standardai.co.kr/health").StatusCode # 기대: 200
(Invoke-WebRequest "https://admin.standardai.co.kr/api/status" `
  -Headers @{ "Authorization"="Bearer <ADMIN_TOKEN>" }).StatusCode # 기대: 200 (토큰 유효 시)
(Invoke-WebRequest "https://admin.standardai.co.kr/api/report").StatusCode # 기대: 200 (리포트 생성 성공 시)
```

위 스크립트는 **1)** Cloud Run 서비스 `quali-admin-domap`에 최신 버전 트래픽 할당, **2)** 웹 브라우저를 열어 캐시를 피한 새 페이지 로드, **3)** PowerShell의 `Invoke-WebRequest`로 헬스체크 및 API 응답 코드를 확인합니다.

<ADMIN\_TOKEN> 부분에는 실제 배포에 사용된 Admin 토큰을 넣어주세요. 각 명령의 결과로 예상되는 HTTP 상태 코드를 주석으로 표시했습니다. 만약 기대와 다른 응답이 나온다면 앞서 설명한 원인별 해결책을 적용하세요.

• **도메인 매핑 확인** - 도메인이 올바른 서비스에 연결됐는지 중간에 확인하고 싶을 때 사용하는 명령어입니다:

```
gcloud beta run domain-mappings describe --domain "admin.standardai.co.kr" --region asia-northeast1 \
--format="value(status.resourceRecords)"
```

위 명령은 해당 도메인의 Cloud Run 매핑 정보를 조회합니다. `status.resourceRecords`에는 도메인 CNAME 등이 나타나며, 별도로 `spec.routeName`을 조회하면 현재 연결된 서비스 이름(`quali-admin-domap`)을 확인할 수 있습니다. 일반적으로 한 번 설정한 도메인 매핑은 바뀌지 않지만, 혹시 문제가 있는 경우 이 정보를 활용해 트러블슈팅합니다.

## 7. Definition of Done (DoD)

마지막으로, 배포 작업이 성공적으로 완료되었는지 확인하는 체크리스트입니다. 아래 사항을 모두 만족하면 배포 완료 (DoD)에 도달했다고 볼 수 있습니다:

- [ ] **서명 커밋**: 모든 커밋이 서명되어 GitHub에서 “Verified”로 표시되었고, main 브랜치에 **서명된 커밋만** 병합되었습니다 (브랜치 보호 규칙 준수).
- [ ] **PR 머지 완료**: 코드 리뷰 승인 및 CI 체크 통과 후 기능 브랜치가 main에 정상 머지되었습니다. GitHub Actions가 자동으로 배포 프로세스를 시작했습니다.
- [ ] **Cloud Run 최신 리비전 활성화**: Cloud Run 콘솔 혹은 CLI를 통해 `quali-admin-domap` 서비스의 **트래픽이 100% 최신 리비전(LATEST)에 할당**되었음을 확인했습니다 <sup>18</sup>. 이전 버전으로의 트래픽 잔류가 없습니다.
- [ ] **스모크 테스트 통과**: 배포된 서비스의 핵심 엔드포인트를 수동 또는 자동으로 테스트하여 모두 **정상 동작**함을 확인했습니다. 예) `/health` → 200 OK, `/api/status` (토큰 포함) → 200 OK, `/api/report` → 200 OK <sup>19</sup>. 필요한 경우 관리자 UI에서 주요 기능을 한 바퀴 테스트합니다.
- [ ] **도메인 정상 서비스**: 사용자용 도메인(`admin.standardai.co.kr`)을 통해 서비스에 접속하여 최신 업데이트가 반영된 것을 확인했습니다. URL 접근, 페이지 로드, 리소스 파일이 오류 없이 제공되며, 브라우저 캐시 문제도 발생하지 않았습니다.

以上的 항목을 모두 만족하면 **"배포 성공"**입니다. 이제 QualiJournal의 운영자는 최신 버전 서비스를 문제없이 이용할 수 있습니다. 배포 과정에서 발생하는 문제는 이 가이드의 절차와 팁을 참고하여 해결하고, 팀 내에 공유하여 모두가 **일관된 워크플로우**를 따르도록 합니다.

1 David Goodwin – David Goodwin

<https://codepoets.co.uk/author/admin/>

2 3 4 5 Adding a new SSH key to your GitHub account - GitHub Docs

<https://docs.github.com/en/authentication/connecting-to-github-with-ssh/adding-a-new-ssh-key-to-your-github-account>

6 7 9 10 11 17 ci.yml

<https://github.com/cdmacs1003-cyber/quali-journal/blob/1f1a913fef6036d0dbb7580446cd003c1b63ff51/.github/workflows/ci.yml>

8 12 13 14 18 deploy-admin-domap.yml

<https://github.com/cdmacs1003-cyber/quali-journal/blob/1f1a913fef6036d0dbb7580446cd003c1b63ff51/.github/workflows/deploy-admin-domap.yml>

15 16 server\_quali.py

[https://github.com/cdmacs1003-cyber/quali-journal/blob/1f1a913fef6036d0dbb7580446cd003c1b63ff51/server\\_quali.py](https://github.com/cdmacs1003-cyber/quali-journal/blob/1f1a913fef6036d0dbb7580446cd003c1b63ff51/server_quali.py)

19 deploy-admin.yml

<https://github.com/cdmacs1003-cyber/quali-journal/blob/1f1a913fef6036d0dbb7580446cd003c1b63ff51/.github/workflows/deploy-admin.yml>